Figure 1.6 A contract test between the provider and mock consumer

Once the contract verification is successful, the consumer can deploy the new user page to an environment suitable for a product demo with more confidence that the demo will go smoothly.

This is a simple example to demonstrate how contract testing works. However, in the real world, things can get much more complicated. Imagine having multiple applications using the User API service with legacy fields and multiple versions knocking about, or a web application communicating to different data services, not just the User API.

For now, the core fundamental to understanding contract testing is that engineers can safely deploy their changes without requiring dedicated test environments. The predefined contract designed before development is now part of the test suite created by the consumer teams. As the contract test is directly connected to the request within the consumer implementation, any changes to the request itself will automatically update the contract. The provider teams will automatically verify the new contract if it's compatible with the existing implementation of their service. Conversely, any changes to the implementation from the provider will check that any change is still compatible with the existing applications using the service.

1.5 Contract testing in action

To guide you with hands-on material, we created an example GitHub project, found at https://github.com/mdcruz/pact-js-example, which contains a comprehensive contract testing suite for web applications, mobile, GraphQL, and events, ready to be run as part of a workflow using GitHub actions. This GitHub project will be referred to from chapter 4 onward in much more detail, but let's look at the coding perspective to see how it all works.